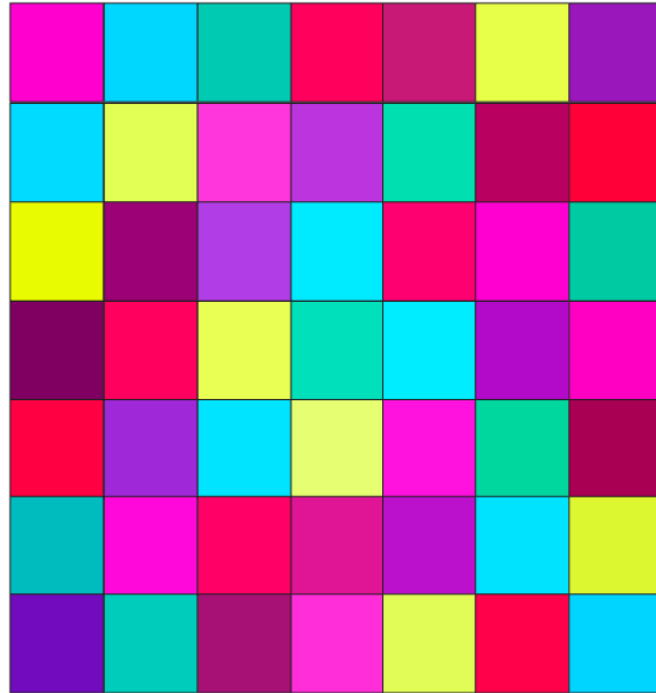


Fisher scoring with Python



Fisher scoring with Python

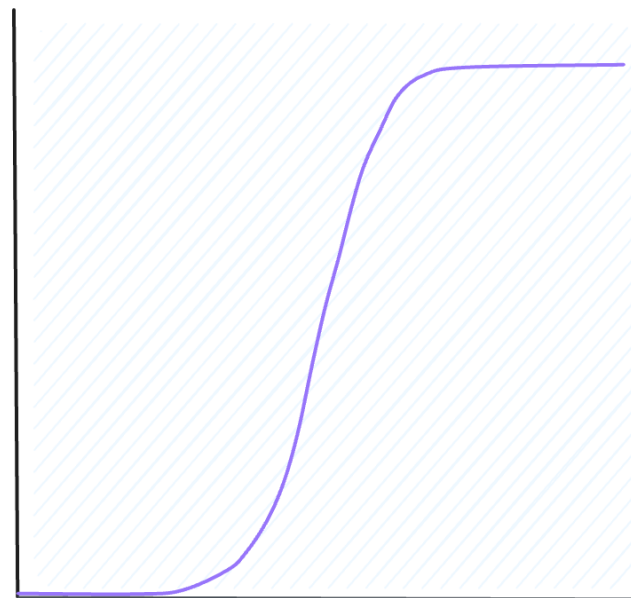


The Fisher Scoring algorithm is an iterative optimization algorithm that updates model parameters using the expected or observed Fisher information matrix.

The `fisher-scoring` package contains Python Scikit-Learn compatible implementations of the Fisher Scoring algorithm for the standard logistic regression model and its generalizations for multi-class and extreme imbalance:`

- **FisherScoringLogisticRegression:** Standard logistic regression using Fisher scoring with statsmodels-like outputs.
- **FisherScoringMultinomialRegression:** Multinomial logistic regression with Fisher Scoring for multi-class.
- **FisherScoringFocalRegression:** Logistic regression with focal loss for imbalanced data with Fisher Scoring.

Logistic Regression



Install with pip:



```
pip install fisher-scoring
```

Fisher scoring with Python



```
from optbinning import BinningProcess
from fisher_scoring_model import
FisherScoringLogisticRegression
from sklearn.pipeline import make_pipeline

# Create a WOE pipeline
binning_process =BinningProcess(
    variable_names=features
)

model = FisherScoringLogisticRegression(
    use_bias=True,
    information='expected',
    verbose=0
)

woe_logistic_model = make_pipeline(
    binning_process,
    model
)

# Fit pipeline
pipeline.fit(X_train, y_train)

# Print summary
woe_logistic_model[-1].display_summary(style='default')
```

Fisher Scoring Logistic Regression Fit

Total Fisher Scoring Iterations: 10
Log Likelihood: -1157.3816
Beta 0 = intercept (bias): True

fisher-scoring

Fisher Scoring Logistic Regression Summary

Parameter	Estimate	Std. Error	Wald Statistic	P-value	Lower CI	Upper CI
intercept (bias)	-2.2571	0.0727	-31.0343	0.0000	-2.3996	-2.1145
revolving_utilization_of_unsecured_lines	-1.0125	0.0515	-19.6430	0.0000	-1.1135	-0.9115
account_never_delinq_percent	-1.0259	0.0523	-19.6299	0.0000	-1.1283	-0.9235
net_fraction_revolving_burden	-0.3907	0.0846	-4.6192	0.0000	-0.5565	-0.2249
external_risk_estimate	-0.7781	0.0718	-10.8446	0.0000	-0.9188	-0.6375
num_total_cc_accounts	-1.2579	0.2734	-4.6011	0.0000	-1.7938	-0.7221
average_months_in_file	-0.4565	0.0987	-4.6266	0.0000	-0.6498	-0.2631

Optimization terminated successfully.
Current function value: 0.165340
Iterations 9

statsmodels

Logit Regression Results

```
=====
Dep. Variable:                y      No. Observations:          7000
Model:                        Logit   Df Residuals:              6993
Method:                       MLE     Df Model:                  6
Date:                         Wed, 31 Jul 2024   Pseudo R-squ.:          0.4914
Time:                         13:17:16     Log-Likelihood:         -1157.4
converged:                     True      LL-Null:                 -2275.6
Covariance Type:               nonrobust   LLR p-value:             0.000
=====
```

	coef	std err	z	P> z	[0.025	0.975]
const	-2.2571	0.073	-31.034	0.000	-2.400	-2.115
revolving_utilization_of_unsecured_lines	-1.0125	0.052	-19.643	0.000	-1.114	-0.911
account_never_delinq_percent	-1.0259	0.052	-19.630	0.000	-1.128	-0.923
net_fraction_revolving_burden	-0.3907	0.085	-4.619	0.000	-0.556	-0.225
external_risk_estimate	-0.7781	0.072	-10.845	0.000	-0.919	-0.637
num_total_cc_accounts	-1.2579	0.273	-4.601	0.000	-1.794	-0.722
average_months_in_file	-0.4565	0.099	-4.627	0.000	-0.650	-0.263

Fisher scoring with Python



Have a look at the examples in notebooks ↩

WOE Logistic Regression: <https://tinyurl.com/m3mk8b8k>

Focal Loss: <https://tinyurl.com/tpbhwhhp2>

Multinomial: <https://tinyurl.com/j8ysedak>

Fisher scoring with Python



Source code is available here ↪

<https://www.github.com/xRiskLab/fisher-scoring>